

Word Embeddings

Word Embeddings

- Is a way to control the size of word representation by limiting it to a fixed sized vector.
- In other words we want to find an embedding for each word in some vector space exhibiting some desired properties.
- Means –
 - if two words are similar in meaning they should be closer to each other compared to words that are not.
 - If two pair of words have similar difference in their meanings, they should be approximately equally separated in the embedded space.
- Such representation can be used to find – synonyms, analogies, identifying concepts around which the words are clustered, classifying words as positive, negative, neutral

Term	Vector representation
I	[-0.04813035, -0.08041322, 0.02042717, -0.04620057, 0.00856122, 0.02766979]
love	[-0.01097935, 0.0055207, -0.02713158, 0.04876678, 0.01179293, 0.02840331]
eating	[-0.00256152, -0.04594067, -0.02137552, 0.05613157, -0.04852077, 0.05093377]
fruits	[-0.03204666, -0.06197819, 0.02622314, -0.01787718, -0.02552203, 0.07250848]
driving	[0.02127126, -0.00173423, -0.04276158, -0.06915958, 0.03542514, -0.03850113]
cars	[0.02993043, -0.03943288, 0.08275513, -0.06427795, 0.07180958, 0.01986287]

Text	Vector representation
I love eating fruits	[-0.02342947, -0.04570285, -0.0004642, 0.01020515, -0.01342216, 0.04487884]
I love driving cars	[-0.02955485, -0.03744626, -0.0033522, 0.00128311, 0.01017707, 0.02803655]

Text	Vector representation
I love eating fruits	[1, 1, 1, 1, 0, 0]
I love driving cars	[1, 1, 0, 0, 1, 1]

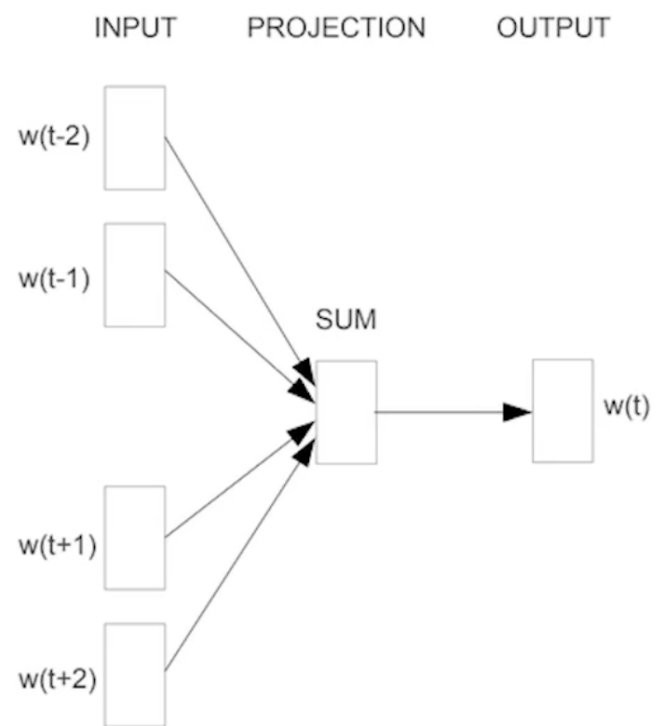
Word2Vec

- Word2Vec is one of the most popular examples of word embeddings used in practice.
- As name indicates, it transforms words to vectors.
- Core idea – A model that is able to predict a given word given neighboring words, or vice versa, predict neighboring words for a given word is likely to capture the “contextual meaning” of words very well.
- These are in fact two flavours of Word2Vec models
 - One where you are given neighboring words called CBOW (continuous bag of words)
 - Other where you are given the middle word called Skip-gram

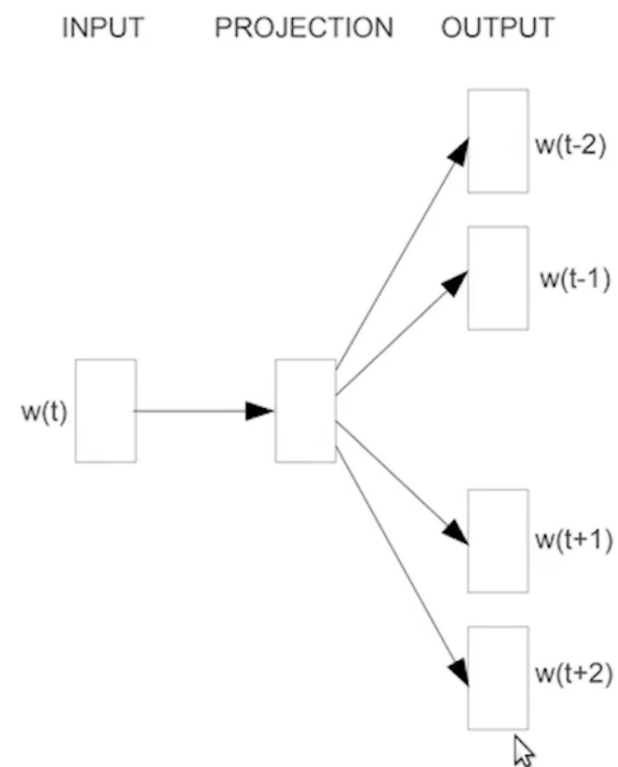
- You pick any word from the sentence
- Convert it into a one-hot encoded vector and feed it into a neural network (or any probabilistic model) that is designed to predict a few surrounding words – its context.
- Using a suitable loss function optimize the weights or parameters of the model and repeat this till it learns to predict the context words as best as it can.
- Take an intermediate representation like a hidden layer in a neural network – the output of that layer for a given word become the corresponding word vector.

Word2Vec properties

- Robust, distributed representation - Yields a very robust representation of words. Why? Because meaning of each word is distributed throughout the vector.
- Vector size is independent of the vocabulary – the size of the vector is up to you. How you want to tune performance vs complexity. It remains constant no matter how many words you train on, unlike Bag of Words for instance – where the size grows with the number of unique words.
- Train once, store in a lookup table. Once you pre-train a large set of word vectors you can use them efficiently without having to transform again.
- Deep learning ready – it is ready to be used in deep learning architectures. It can be used as the input vector for RNNs. It is also possible to use RNNs to learn even better word embeddings. Some other optimization are possible that further reduces the model and training complexity such as representing the output words using Hierarchical SoftMax, computing loss using Sparse Cross Entropy, etc.



CBOW



Skip-gram

A picture is worth a thousand words

Continuous Bag of Words:

Input = ["picture", "is", "a", "thousand"], Output = "worth"

Skip-gram:

Input = "worth", Output = ["picture", "is", "a", "thousand"]

GloVe

- Is an extension to word2vec.
- Disadvantage of Word2Vec – takes only “local” context into consideration while learning embedding
- For example “Ice cream is in the fridge”. It does not capture whether the word “ice” occurs in context of word like this is a random occurrence. In other words – fail to capture the global statistics of the word.
- GloVe take advantage of global statistics in addition to the local context
- Global statistic is derived using word to word cooccurrence matrix.
 - If we have “n” words in the vocabulary, then co-occurrence matrix is a “nxn” matrix – that captures how frequently two words occur together in a given corpus.

GloVe

- GloVe – Global Vector for word representation tries to directly optimize the vector representation of each word using co-occurrence statistics.
(Word2Vec sets up an ancillary prediction task – distribution does not depend on the words in the document)
- The probability that word “j” appears in the context of word “i” is computed. $P(j|i)$ for all the word pairs ij in a given corpus. When we say j appears in the context of word i , is that word j is present in the vicinity of word i , either right next to it or a few words away.
- We count all such occurrences of words i and j in our text collection, and then normalize account to get probability.
- Then two random vector is initialized for each word – one for the word when it is acting as a context, and other when it is acting as a target.

GloVe

- For any pair of words ij we want the dot product of the word vectors $w_i \cdot w_j$ to be equal to their co-occurrence probability. Using this as a goal and suitable loss function we can iteratively optimize these word vectors.
- The result is a set of vectors that capture the similarities and differences between individual words.
- In other words we are factorizing the co-occurrence probability matrix into two smaller matrices.

Why Cooccurrence Probabilities?

	<i>solid</i>	<i>water</i>
 <i>ice</i>	$P(\text{solid} \text{ice})$	$P(\text{water} \text{ice})$
 <i>steam</i>	$P(\text{solid} \text{steam})$	$P(\text{water} \text{steam})$
$\frac{P(\text{solid} \text{ice})}{P(\text{solid} \text{steam})} \gg 1$		$\frac{P(\text{water} \text{ice})}{P(\text{water} \text{steam})} \approx 1$

	play	love	football	I	cricket
play	0.0	0.0	0.0	1.0	1.0
love	0.0	0.0	1.0	2.0	1.0
football	0.0	1.0	0.0	0.0	0.0
I	1.0	2.0	0.0	0.0	0.0
cricket	1.0	1.0	0.0	0.0	0.0

$$P(\text{cricket}/\text{play}) = 1$$

$$P(\text{cricket}/\text{love}) = 0.5$$

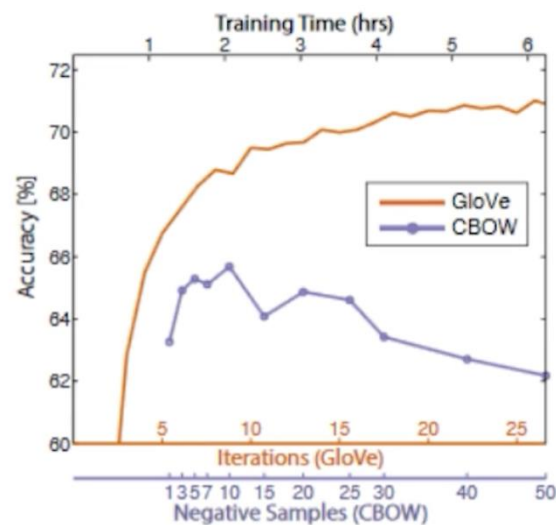
$$P(\text{cricket}/\text{play}) / P(\text{cricket}/\text{love}) = 2$$

(Cricket is more relevant to "play" than "love", and if the probability is closer to one – both the words – "play" and "love" are relevant to "cricket".

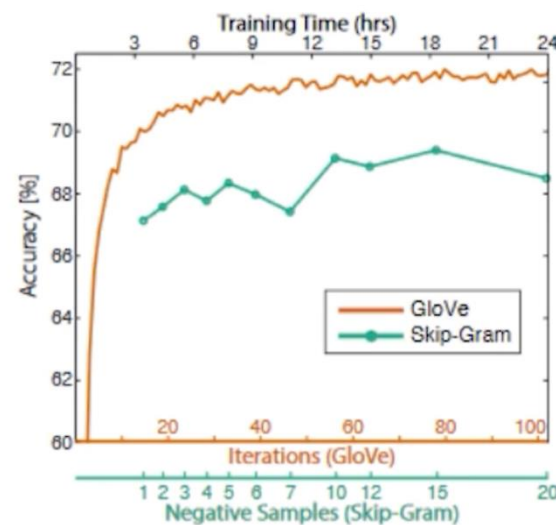
Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)/P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

Ice is more relevant to solid than steam, whereas steam is more relevant to gas than ice. Both are relevant to the word water, irrelevant to the word fashion. Therefore, we can derive the relationship between the words using simple statistics.

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$



(a) GloVe vs CBOW



(b) GloVe vs Skip-Gram

Figure 4: Overall accuracy on the word analogy task as a function of training time, which is governed by the number of iterations for GloVe and by the number of negative samples for CBOW (a) and skip-gram (b). In all cases, we train 300-dimensional vectors on the same 6B token corpus (Wikipedia 2014 + Gigaword 5) with the same 400,000 word vocabulary, and use a symmetric context window of size 10.